Tuto : créer son système multi-agents avec Claude Code

Oct 18, 2026 · @BigBen

1. Introduction et principes clés

Ce tuto explique comment construire, dans Claude Code, une équipe d'agents qui produit des sites web : un orchestrateur distribue le travail, des agents spécialisés produisent, d'autres vérifient. Il regroupe l'architecture, les étapes, des modèles de fichiers prêts à copier, les skills à créer, et les conseils pour tenir dans un abonnement Pro.

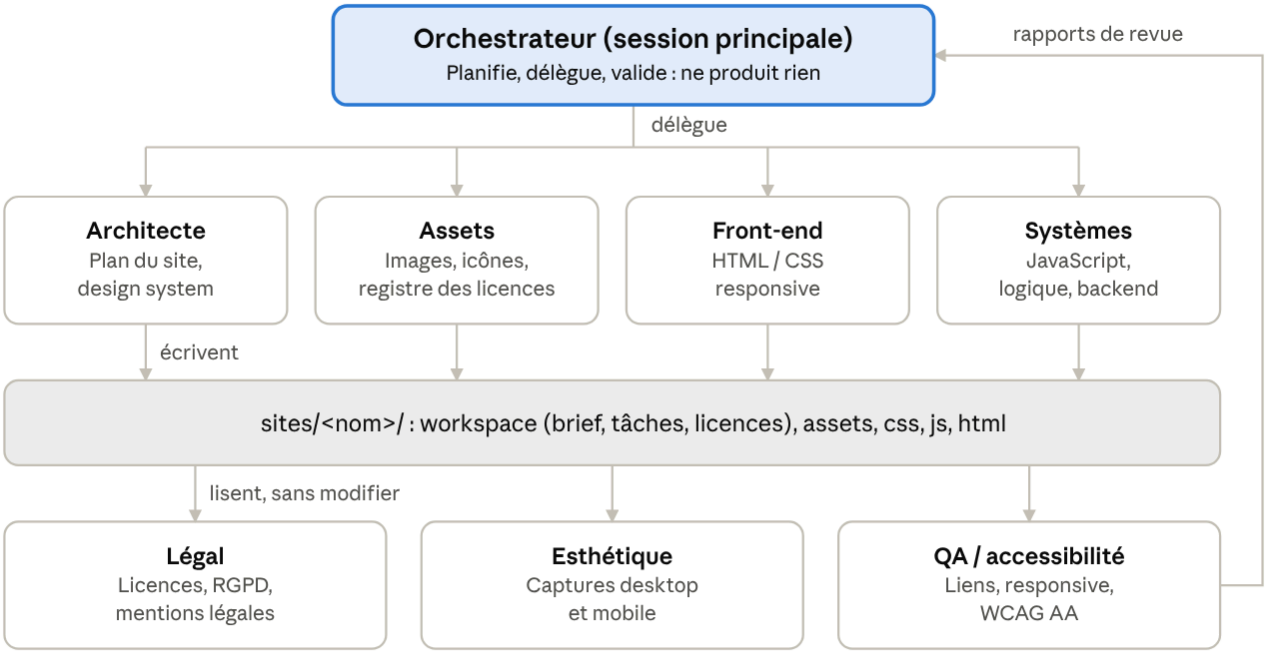
Quatre principes guident tout le reste :

1. **Séparer producteurs et reviewers.** Un agent qui vérifie son propre travail le valide presque toujours. Les vérifications (légal, esthétique, QA) sont faites par des agents qui n'ont rien produit et qui n'ont pas le droit de modifier le code.
2. **L'orchestrateur ne produit rien.** Il découpe, délègue, collecte et décide. Dans Claude Code, un sous-agent ne peut pas lancer d'autres sous-agents : l'orchestrateur est donc la session principale, guidée par le fichier `CLAUDE.md`.
3. **Les agents communiquent par fichiers.** Chaque sous-agent démarre sans mémoire de la conversation. Un brief, une liste de tâches et un registre des assets partagés dans un dossier `workspace/` rendent le système fiable.
4. **Commencer petit.** Quatre agents d'abord, puis on sépare ou on ajoute quand un problème revient. Chaque agent coûte des tokens et ajoute un risque de malentendu.

2. Architecture

L'orchestrateur délègue à quatre producteurs qui écrivent dans un dossier partagé ; trois reviewers le lisent sans rien modifier et renvoient leurs rapports à l'orchestrateur.

Les producteurs écrivent, les reviewers lisent, l'orchestrateur arbitre



Un point BLOQUANT renvoie la tâche au producteur, 3 allers-retours maximum

équipe d'agents · 1 orchestrateur, 4 producteurs, 3 reviewers

Les flèches suivent le travail : délégation vers le bas, rapports qui remontent par la droite.

Agent	Rôle	Outils
Orchestrateur	Planifie, délègue, arbitre	Tous, dont Agent
Architecte	Arborescence, design system, découpage technique	Read, Write, Glob
Assets	Trouve, télécharge, optimise, renomme ; remplit le registre	WebSearch, WebFetch, Bash, Read, Write
Front-end	HTML/CSS responsive selon le design system	Read, Write, Edit, Glob
Systèmes	JavaScript, formulaires, API, logique, backend	Read, Write, Edit, Bash
Légal	Licences, polices, mentions légales, RGPD, cookies	Read, Glob, WebFetch
Esthétique	Cohérence visuelle, hiérarchie, espacements, contrastes	Read (captures d'écran)
QA	Liens cassés, responsive, accessibilité, performance	Read, Bash

Version allégée pour démarrer (4 agents)

Sur un abonnement Pro, commence avec cette version, puis sépare les rôles quand le besoin apparaît.

- **Orchestrateur** : la session principale.
- **Assets** : images et registre des licences.
- **Front-end** : HTML, CSS et le peu de JavaScript nécessaire (fusionne architecte et systèmes au début).
- **Reviewer unique** : légal, esthétique et QA dans un seul rapport, avec une section par thème.

3. Les étapes de A à Z

Suivre ces étapes dans l'ordre ; un premier système utilisable tient en une ou deux sessions.

1. **Choisir un projet test.** Un petit projet concret, par exemple un site vitrine de 3 pages. C'est en le faisant tourner qu'on voit ce qui coince.
2. **Créer la structure du projet** (arborescence ci-dessous).

3. **Écrire le `CLAUDE.md` de l'orchestrateur** : le processus, les règles de délégation, la limite d'allers-retours (modèle en section 4).
4. **Créer chaque agent** dans `.claude/agents/`, un fichier `.md` par agent, avec nom, description, outils et modèle.
5. **Créer les skills** dans `.claude/skills/<nom>/SKILL.md` : les savoir-faire partagés entre agents (section 5).
6. **Préparer le _template/ et ses fichiers de travail** dans `workspace/` : `brief.md`, `tasks.md`, `design-system.md`, `assets-registry.md`.
7. **Équiper les reviewers.** Installer Playwright pour capturer la page en desktop et en mobile : l'agent esthétique juge des captures, pas du CSS.
8. **Lancer le projet test** et tout observer : qui a mal compris son rôle, quels rapports sont trop vagues, où partent les tokens.
9. **Corriger** descriptions, skills et formats de sortie. Après 2 ou 3 projets, les instructions se stabilisent.
10. **Ajouter des agents seulement si besoin** (SEO, rédaction, performance), quand un problème revient souvent.
11. **Créer un template de départ** à copier pour chaque nouveau projet : structure, design system de base, scripts.

Arborescence

Les agents et les skills sont communs ; chaque site vit dans son propre dossier `sites/<nom-du-site>/`, avec son `workspace/`, ses images et son code. Tu lances toujours Claude Code depuis le dossier `agence/`.

```
agence/                          ← lancer Claude Code ici
├── CLAUDE.md                    ← instructions de l'orchestrateur
├── README.md
├── .gitignore
├── .claude/
│   ├── settings.json            ← permissions (dossiers interdits en
lecture)
│   ├── agents/                  ← les agents, communs à tous les sites
│   └── skills/                  ← les skills, communes
├── scripts/                     ← optimisation d'images, captures
├── _template/                   ← squelette copié pour chaque nouveau site
│   ├── workspace/
│   │   ├── brief.md
│   │   ├── tasks.md
│   │   ├── design-system.md
│   │   ├── assets-registry.md
│   │   └── reviews/
```

```
|   ├── assets/
|   |   ├── images/
|   |   └── icons/
|   ├── css/main.css
|   ├── js/main.js
|   └── index.html
└── sites/
    ├── boulangerie-martin/      ← un dossier par site
    |   ├── workspace/           ← brief, tâches, licences, rapports DE CE
SITE
    |   ├── assets/images/       ← images DE CE SITE
    |   ├── css/
    |   ├── js/
    |   └── index.html
    └── portfolio-julie/
```

Le `workspace/` est dans chaque site, jamais à la racine : sinon les tâches et les licences de deux sites se mélangent. Avantages : sites isolés, agents qui ne lisent qu'un dossier (moins de tokens), démarrage instantané grâce au `_template/`, et un dossier de site livrable ou déployable tel quel.

Prompt pour créer l'architecture

À coller dans Claude Code, dans le dossier où créer `agence/`. Il crée la structure sans écrire le contenu des agents, des skills ni du `CLAUDE.md`.

```
Crée uniquement l'architecture de dossiers et les fichiers de départ d'un
système multi-agents de création de sites web. N'écris PAS le contenu de
CLAUDE.md, des agents ni des skills : je les rédigerai moi-même. Crée-les
vides ou avec une simple ligne de titre.

Crée cette structure dans un dossier "agence/" :

agence/
├── CLAUDE.md                     (vide, juste "# Orchestrateur")
├── .claude/
|   ├── settings.json             (JSON valide minimal : {} )
|   ├── agents/                   (dossier vide avec un .gitkeep)
|   └── skills/                   (dossier vide avec un .gitkeep)
├── scripts/                      (dossier vide avec un .gitkeep)
├── _template/
|   ├── workspace/
|   |   ├── brief.md              (titre "# Brief" + sections vides :
Objectif, Public cible, Pages, Style souhaité, Contraintes)
|   |   ├── tasks.md              (titre + tableau vide : id | tâche | agent
| dépend de | statut)
```

```
|   |   ├── design-system.md       (titre + sections vides : Couleurs,
Typographie, Espacements, Composants)
|   |   ├── assets-registry.md     (titre + tableau vide : fichier | source
(URL) | licence | auteur | usage commercial | modifié)
|   |   └── reviews/               (.gitkeep)
|   ├── assets/
|   |   ├── images/                (.gitkeep)
|   |   └── icons/                 (.gitkeep)
|   ├── css/
|   |   └── main.css               (commentaire "/* Styles du site */")
|   ├── js/
|   |   └── main.js                (commentaire "// Scripts du site")
|   └── index.html                 (squelette HTML5 minimal en français :
doctype, lang="fr", meta charset et viewport, title vide, lien vers
css/main.css, script js/main.js en defer)
└── sites/                         (dossier vide avec un .gitkeep)

Ajoute aussi à la racine :
- un .gitignore (node_modules/, .DS_Store, *.log, dist/)
- un README.md court qui explique la structure : agents et skills communs
dans .claude/, _template/ copié pour chaque nouveau site dans sites/<nom-du-
site>/, chaque site a son propre workspace/ et ses propres assets.

Quand c'est fini, affiche l'arborescence avec tree (ou find si tree n'est pas
installé) et ne fais rien d'autre.
```

Ensuite, relance Claude Code depuis `agence/` pour qu'il charge le `CLAUDE.md` et les agents, et vérifie-les avec `/agents`. Pour tester : « Crée un nouveau site "Boulangerie Martin" : site vitrine de 3 pages. »

Règles pour écrire un agent

- La **description** est essentielle : c'est elle qui dit à l'orchestrateur quand utiliser l'agent.
- Le **minimum d'outils** nécessaire ; les reviewers n'ont ni `Write` ni `Edit`.
- Un **format de sortie imposé** : quel fichier, quelle structure, quels niveaux de gravité.
- Un **modèle adapté** à la difficulté de la tâche (section 6).
- **Un dossier ou fichier par agent**, pour que deux agents n'écrivent jamais au même endroit.

4. Modèles de fichiers à copier

Ces modèles sont des points de départ : adapte noms, modèles et règles à ton projet.

CLAUDE.md (orchestrateur)

```
# Orchestrateur

Tu coordonnes une équipe d'agents. Tu ne produis ni code ni asset toi-même.

## Démarrer un nouveau site
1. Demande-moi le nom du site s'il n'est pas donné.
2. Convertis-le en nom de dossier : minuscules, tirets, sans accents
   (ex. « Boulangerie Martin » → boulangerie-martin).
3. Si sites/<nom>/ existe déjà, demande-moi si je veux le reprendre ou en
créer un autre.
4. Copie _template/ vers sites/<nom>/.
5. Écris le brief dans sites/<nom>/workspace/brief.md.

## Règle des chemins
- SITE = sites/<nom>/ . Donne ce chemin à chaque agent dans chaque
délégation.
- Les agents ne lisent et n'écrivent QUE dans SITE (plus .claude/skills et
scripts/).
- Images : SITE/assets/images/ — icônes : SITE/assets/icons/
- Registre des licences : SITE/workspace/assets-registry.md
- Rapports : SITE/workspace/reviews/

## Processus
1. Délègue à `architect` : plan du site + SITE/workspace/design-system.md.
2. Écris SITE/workspace/tasks.md (une ligne par tâche : id, agent, dépend de,
statut).
3. Lance les producteurs dans l'ordre des dépendances, un à la fois.
4. Quand un livrable est complet, lance `reviewer` (ou les reviewers
séparés).
5. Un point BLOQUANT dans SITE/workspace/reviews/ → renvoie la tâche au
producteur.
   3 allers-retours maximum, puis demande-moi.
6. Termine par un résumé de 10 lignes maximum.

## Règles
- Donne à chaque agent seulement les fichiers dont il a besoin, avec leurs
chemins.
- Réponses brèves. Pas de récapitulatif. Ne réaffiche jamais du code complet.
- Mets à jour le statut dans tasks.md après chaque délégation.
```

Agent producteur : assets

```
---
name: asset-maker
description: Trouve, télécharge, optimise et enregistre les images et icônes
du site. À utiliser quand une tâche demande des visuels.
tools: WebSearch, WebFetch, Bash, Read, Write
model: haiku
---
Tu travailles uniquement dans le dossier SITE indiqué par l'orchestrateur
(ex. sites/boulangerie-martin/). Ne touche jamais aux autres dossiers de
sites/.

1. Lis la tâche et SITE/workspace/design-system.md.
2. Cherche uniquement sur les sources autorisées de la skill image-sourcing.
   Si aucune image libre ne convient, crée une illustration SVG simple.
3. Optimise avec scripts/optimize-images.sh (ne traite pas les images une par
une).
4. Enregistre dans SITE/assets/images/ (icônes : SITE/assets/icons/), noms en
kebab-case.
5. Ajoute une ligne par fichier dans SITE/workspace/assets-registry.md.

Réponds en 3 lignes : nombre d'images, fichiers créés, problèmes éventuels.
```

Agent producteur : front-end

```
---
name: frontend-dev
description: Écrit le HTML et le CSS du site à partir du plan et du design
system. À utiliser pour toute tâche de mise en page ou de style.
tools: Read, Write, Edit, Glob
model: sonnet
---
Tu travailles uniquement dans le dossier SITE indiqué par l'orchestrateur
(ex. sites/boulangerie-martin/). Ne touche jamais aux autres dossiers de
sites/.

- HTML dans SITE/ (index.html et autres pages), styles dans SITE/css/.
- Suis les skills html-css-standards et design-system.
- Utilise uniquement les variables CSS de SITE/workspace/design-system.md.
- Mobile-first, HTML sémantique, texte alternatif sur chaque image.
- Images référencées depuis assets/images/ (chemins relatifs).
- Modifie seulement les fichiers de ta tâche.

Réponds en 3 lignes : fichiers modifiés, ce qui reste à faire.
```

Agent reviewer : légal

```
---
name: legal-reviewer
description: Vérifie la légalité des assets, polices et mentions du site. À
utiliser après la production, avant la livraison.
tools: Read, Glob, WebFetch
model: haiku
---
Tu es un relecteur juridique pour des sites web en France/UE.
Tu travailles uniquement dans le dossier SITE indiqué par l'orchestrateur.
Suis la skill legal-checklist-fr et la skill review-report-format.

Vérifie :
1. Chaque fichier de SITE/assets/ figure dans SITE/workspace/assets-
registry.md (source, licence, auteur).
2. Les licences autorisent l'usage commercial et la modification.
3. Les polices sont sous licence libre (OFL, Google Fonts...).
4. Mentions légales (LCEN), politique de confidentialité, bandeau cookies si
nécessaire (RGPD).

Tu NE modifies AUCUN fichier. Écris ton rapport dans
SITE/workspace/reviews/legal.md.
```

sites/<nom>/workspace/tasks.md

```
| id | tâche | agent | dépend de | statut |
|----|-------|-------|-----------|--------|
| T1 | Plan + design system | architect | - | fait |
| T2 | Images page d'accueil | asset-maker | T1 | en cours |
| T3 | HTML/CSS accueil | frontend-dev | T1, T2 | à faire |
| T4 | Formulaire de contact | systems-dev | T3 | à faire |
| T5 | Revue complète | reviewer | T3, T4 | à faire |
```

sites/<nom>/workspace/assets-registry.md

```
| fichier | source (URL) | licence | auteur | usage commercial | modifié |
|---------|--------------|---------|--------|------------------|---------|
| hero-montagne.webp | https://unsplash.com/... | Unsplash | J. Dupont | oui
| recadré |
| icone-contact.svg | créé par asset-maker | - | - | oui | - |
```

Format commun des rapports de revue

```
# Revue <légal | esthétique | QA> — <date>
Verdict : VALIDÉ | À CORRIGER

| gravité | fichier | problème | correction proposée |
|---------|---------|----------|---------------------|
| BLOQUANT | site/index.html | Image sans licence | Remplacer ou documenter |
| À AMÉLIORER | site/css/main.css | Contraste du bouton 3,1:1 | Foncer à
4,5:1 |
```

5. Les skills à créer

Une skill est un savoir-faire écrit une fois et partagé par plusieurs agents ; elle n'est chargée que quand elle sert, contrairement au `CLAUDE.md` lu à chaque message. Chaque skill est un dossier `.claude/skills/<nom>/` contenant un `SKILL.md`, et éventuellement des scripts.

Skill	Contenu	Utilisée par
image-sourcing	Sites autorisés (Unsplash, Pexels, Wikimedia en CC0/CC-BY), règles de nommage, format du registre	Assets, Légal
image-optimization	Script de conversion WebP/AVIF, tailles responsive, compression (sharp ou ImageMagick)	Assets
design-system	Lire et appliquer design-system.md : variables CSS, grille, espacements	Architecte, Front-end, Esthétique
html-css-standards	HTML sémantique, mobile-first, conventions CSS, pas de styles inline	Front-end, QA
legal-checklist-fr	LCEN, RGPD, cookies (CNIL), types de licences et ce qu'elles autorisent	Légal
aesthetic-review	Grille : hiérarchie, contraste, alignement, cohérence, densité	Esthétique
accessibility-wcag	Checklist WCAG AA : textes alternatifs, contrastes, navigation clavier	QA, Front-end
review-report-format	Format commun des rapports (BLOQUANT / À AMÉLIORER / OK)	Tous les reviewers

Exemple : image-sourcing/SKILL.md

```
---
name: image-sourcing
description: Règles pour trouver et documenter des images libres de droits. À
utiliser avant tout téléchargement d'image.
---
# Sources autorisées
- unsplash.com, pexels.com : usage commercial autorisé, pas de revente brute.
- commons.wikimedia.org : uniquement CC0, CC-BY ou CC-BY-SA (noter l'auteur).
- Interdit : Google Images, Pinterest, sites de presse, images sans licence
visible.

# Nommage
<page>-<sujet>.webp en kebab-case, ex. accueil-equipe.webp

# Registre (obligatoire)
Une ligne par fichier dans workspace/assets-registry.md :
fichier | URL source | licence | auteur | usage commercial | modifié

# Si rien ne convient
Créer une illustration SVG simple aux couleurs du design system.
```

Ce que je mets dans une skill plutôt que dans un agent : toute règle qui sert à plus d'un agent, ou qui est longue. L'agent ne garde que son rôle, ses étapes et son format de sortie.

6. Abonnement Pro et choix des modèles

Sur Pro, un système multi-agents tient si on choisit un modèle par agent et qu'on évite le parallèle massif.

Comment fonctionnent les limites

- **Un seul quota** pour Claude (le chat) et Claude Code : tout compte au même endroit (Anthropic).
- **Deux fenêtres** : une fenêtre glissante de 5 heures et un plafond hebdomadaire (morphllm).
- **Le parallèle coûte cher** : plusieurs instances de Claude Code en même temps épuisent les limites plus vite (Anthropic).
- **Limite atteinte** : attendre la réinitialisation. Claude Code peut proposer de continuer avec des crédits API, facturés au tarif API ; il suffit de refuser pour rester dans le forfait.

- **Au-delà** : les forfaits Max offrent 5 fois l'usage du Pro (100 $/mois) ou 20 fois (200 $/mois) (fast.io).

Un modèle par agent

Le modèle se règle dans l'en-tête du fichier de l'agent : `model: opus`, `sonnet`, `haiku` ou `inherit`. Tape `/model` dans Claude Code pour voir les modèles accessibles avec ton forfait ; selon le forfait, Opus peut être indisponible ou consommer le quota beaucoup plus vite.

Agent	Modèle conseillé	Pourquoi
Orchestrateur	Sonnet	Planifie et délègue, pas besoin du plus puissant
Architecte	Opus si disponible, sinon Sonnet	Une erreur de plan se répercute partout
Systèmes (JS, logique)	Opus si disponible, sinon Sonnet	La qualité du code compte le plus ici
Front-end HTML/CSS	Sonnet	Très bon en HTML/CSS, bien moins coûteux
Assets	Haiku	Recherche, téléchargement, renommage : mécanique
Légal	Haiku	Suffit avec une checklist précise
Esthétique	Sonnet	Analyse des captures avec jugement
QA	Haiku	Exécute des scripts et lit les résultats

7. Économiser les tokens

Le trio le plus rentable : `/clear` entre les tâches, un `CLAUDE.md` court avec les détails dans des skills, et des demandes précises avec les chemins des fichiers.

Gérer le contexte

- **`/clear`** entre deux tâches sans rapport : à chaque message, Claude relit tout l'historique de la session.
- **`/compact`** quand une session s'allonge, en précisant quoi garder (`/compact garde les décisions de design et tasks.md`).
- **Une session par tâche** plutôt qu'une session géante toute la journée.

Alléger ce qui est toujours chargé

- **`CLAUDE.md` sous 100 lignes** : il est relu à chaque message. Les détails vont dans des skills.
- **Désactiver les serveurs MCP, connecteurs et plugins inutiles** : leurs descriptions d'outils occupent du contexte même sans usage.
- **Interdire la lecture des dossiers lourds** dans `.claude/settings.json` (règles `deny`) : `node_modules`, `dist`, images, logs, gros JSON.

Bien demander

- **Demandes précises** avec chemins et résultat attendu : « dans `site/css/header.css`, réduis l'espacement du menu sur mobile ».
- **Mode plan** (Shift+Tab) pour les grosses tâches : corriger un plan coûte peu, recoder une mauvaise direction coûte cher.
- **Moins de réflexion** pour les tâches simples ; garder la réflexion poussée pour l'architecture et les bugs difficiles.
- **Réponses courtes** imposées dans `CLAUDE.md` : pas de récapitulatif, pas de code complet réaffiché.
- **Modifications ciblées** plutôt que réécriture de fichiers entiers.

Organiser les agents

- **Scripts pour le travail mécanique** : optimiser 30 images avec un script ne coûte presque rien.
- **Ne pas lancer tous les agents** à chaque petite correction.
- **Reviews regroupées** sur un livrable complet, une seule fois.
- **Fichiers courts pour chaque agent** : seulement le brief et ce qui le concerne.
- **2 ou 3 allers-retours maximum** entre producteur et reviewer.
- **Agents en séquence** plutôt que 5 en parallèle sur Pro.
- **Template de départ** réutilisé à chaque projet.

Organiser ses heures

- La fenêtre de 5 heures démarre au **premier message** : une petite tâche le matin lance le compteur, et la fenêtre suivante arrive plus tôt l'après-midi.
- Vérifier où en est le quota avec `/usage` ou `/status` (selon ta version) avant un gros travail multi-agents.
- Garder le chat avec Claude pour des questions courtes pendant que les agents travaillent : tout puise dans le même quota.

8. Pièges à éviter et checklist

Pièges

- **Téléchargement bloqué** : selon l'environnement, le réseau peut être restreint. Prévoir la génération SVG ou le dépôt manuel des images.
- **Boucles infinies** entre producteur et reviewer : toujours une limite d'itérations.
- **Écritures concurrentes** : deux agents dans le même fichier en parallèle. Un dossier ou fichier par agent.
- **Reviewer qui corrige lui-même** : il écrit un rapport, le producteur corrige.
- **Descriptions d'agents vagues** : l'orchestrateur choisit le mauvais agent ou n'en choisit aucun.
- **L'agent légal n'est pas un avocat** : il repère les problèmes évidents. Pour un site commercial, faire valider les mentions légales par un professionnel.

Checklist de mise en route

- [ ] Projet test choisi et `workspace/brief.md` écrit
- [ ] Arborescence créée
- [ ] `CLAUDE.md` de l'orchestrateur écrit (moins de 100 lignes)
- [ ] 4 agents de départ créés : architecte, assets, front-end, reviewer
- [ ] Modèle choisi pour chaque agent et vérifié avec `/model`
- [ ] Skills de base créées : image-sourcing, design-system, html-css-standards, review-report-format
- [ ] Script d'optimisation d'images et script de captures (Playwright) en place
- [ ] Dossiers lourds interdits en lecture dans `.claude/settings.json`
- [ ] Premier projet test lancé et observé
- [ ] Descriptions, skills et formats corrigés après le test
- [ ] Template de départ enregistré pour les projets suivants